

Doctorly Technical Call — Walkthrough and Q&A

The Walkthrough (12-15 minutes)

Time split: 12-15 minutes walkthrough, 25-30 minutes questions, 5 minutes your questions to them.

Opening — 1 minute

"I built a REST API for managing calendar events in a doctor's practice. The core requirements were events with attendees, status management, notifications, and persistence. I used ASP.NET Core with EF Core and SQLite. The solution is structured around DDD principles — the domain owns the business rules, infrastructure handles persistence, and features are organised as vertical slices."

Folder structure — 2 minutes

"There are three top-level concerns. Domain contains the entities, enums, interfaces, and exceptions — no dependencies on anything outside itself. Infrastructure contains EF Core, the repository implementations, and the notification stub. Features contains one folder per operation — CreateEvent, GetEvents, UpdateEvent and so on. Each folder has its own controller and request. No shared service classes, no application layer — at this scale that would be ceremony without value."

Domain — 3 minutes

"CalendarEvent is the aggregate root. It has private setters — state can only change through domain methods: Create, Update, Cancel, Complete, AddAttendee. The factory method Create validates inputs before the object exists — you cannot construct an invalid event. Attendee is owned by CalendarEvent. You cannot create an Attendee directly — Attendee.Create is internal and only called from CalendarEvent.AddAttendee. This enforces the aggregate boundary. AddAttendee guards against duplicates and against adding attendees to a cancelled event. These rules live on the entity, not in the controller."

Infrastructure — 2 minutes

"AppDbContext uses IEntityTypeConfiguration to keep mapping out of the entity classes. RowVersion is a Guid we generate on every write, configured as IsConcurrencyToken. SQLite doesn't support the native rowversion type so we manage it ourselves. The repository implements ICalendarEventRepository. GetAll fetches results then orders in memory because SQLite doesn't support ORDER BY on DateTimeOffset columns."

Features — 2 minutes

"Each controller handles exactly one operation. UpdateEvent checks RowVersion before saving — if the client's token doesn't match the database, we return 409 Conflict. AddAttendee does the same. Cancel deliberately does not check RowVersion — cancelling an event that someone else just modified is fine, the intent is clear. INotificationService is called after every write. The implementation is a logging stub — swapping to email requires no changes to the domain or controllers, just a new class and one line in Program.cs."

Tests — 1 minute

"Tests cover the domain invariants — the duplicate attendee guard, the cancelled event guards, the end-before-start guard. No mocks, no EF Core, no HTTP. The domain has no dependencies so there's nothing to mock. I used Shouldly because the failure messages include the variable name and read as full sentences, which makes a failing test self-explanatory without reading the code."

Close — 1 minute

"The main trade-offs I'd flag: SQLite is great for portability but has two limitations I worked around — no native rowversion and no DateTimeOffset ORDER BY. Moving to SQL Server removes both. There's no auth, no pagination, and notifications are stubbed. Those are all listed in the README as the next things I'd add."

Anticipated Questions and Answers

Why SQLite and not SQL Server?

No setup required. Clone and run — the database creates itself on first startup via db.Database.Migrate(). For a time-boxed assessment that matters. In production the only changes are the connection string and swapping the EF Core provider package. The domain and repository don't care.

Walk me through how a Create request flows end to end

The request hits CreateEventController. It calls CalendarEvent.Create with the validated inputs — that's where the end-before-start guard runs. For each attendee in the request it calls AddAttendee, which checks for duplicates and cancelled status. The repository persists it via EF Core. Then INotificationService.NotifyEventCreatedAsync is called — currently logs to console. The controller maps the saved entity to a CalendarEventResponse using the shared CalendarEventMapper and returns 201.

Why is Attendee not an aggregate root?

An attendee has no meaning without an event. You never query attendees independently — you always go through the event. If Attendee had its own repository you could call AttendeeRepository.Add directly and bypass the duplicate check and the cancelled-event

guard in `AddAttendee`. Making it internal to the aggregate forces all creation through the rules.

Why does Cancel not check RowVersion but Update does?

Deliberate. If a doctor cancels an event while someone else is updating the description, the cancellation should still go through — the intent is unambiguous. For Update it matters because you'd be overwriting someone else's changes silently. The asymmetry reflects the business intent, not an oversight.

Why is INotificationService in Domain and not Infrastructure?

The domain defines what it needs. It needs to know that when an event is created, something should be notified. It doesn't need to know whether that's an email, an SMS, or a message queue. Putting the interface in Domain means the domain depends on an abstraction it owns. The concrete implementation sits in Infrastructure. This is the Dependency Inversion principle.

Why no MediatR?

MediatR earns its place when controllers need to coordinate multiple services, fire domain events, or when you want cross-cutting behaviour like logging and validation as pipeline behaviours. Here the controllers are thin — they call one repository method and one notification. Adding MediatR would mean a handler class per operation with no behaviour change. I wanted to show I know when a pattern adds value versus when it adds indirection.

Two doctors update the same event at the same time — walk through exactly what happens

Both load the event and receive the same `RowVersion`, say `abc-123`. Doctor A saves first — the repository calls `UpdateAsync`, EF Core issues `UPDATE ... WHERE RowVersion = 'abc-123'`, it succeeds, and the entity gets a new `RowVersion` `def-456`. Doctor B then submits their update. Their request carries `abc-123`. In `UpdateEventController` we load the current entity — its `RowVersion` is now `def-456`. We compare: `def-456 != abc-123`, return `409 Conflict` with the message "The event was modified by someone else. Please reload and try again." Doctor B reloads, gets the current state with `def-456`, and retries with that token.

Why is Attendee.Create() internal?

Internal means it's only accessible within the same assembly. `CalendarEvent.AddAttendee` calls it. Nothing outside the assembly can call it directly. This enforces the aggregate boundary at the compiler level — you physically cannot create an `Attendee` without going through `CalendarEvent`.

No authentication — how would you add it?

Add JWT bearer authentication in `Program.cs` with `builder.Services.AddAuthentication().AddJwtBearer()`. Add the `[Authorize]` attribute to

controllers. The token payload would carry the user's identity and role — doctor, receptionist, admin. Business rules would then check whether the caller owns or has access to the event they're modifying.

How would you paginate GetAll?

Add page and pageSize query parameters. In the repository use `.Skip((page - 1) * pageSize).Take(pageSize)`. Return a wrapper with TotalCount, Page, PageSize, and the data array. This also fixes the SQLite DateTimeOffset ORDER BY limitation — ordering a page of 20 records in memory is trivial. On SQL Server the ORDER BY runs in the database.

How would you move to SQL Server?

Two changes. Swap UseSqlite for UseSqlServer in Program.cs with the new connection string. Add the Microsoft.EntityFrameworkCore.SqlServer package. RowVersion could then be changed from a Guid we manage to a native byte[] rowversion that SQL Server manages automatically — the column increments on every write without application code.

What would you test next?

Integration tests on the HTTP layer using WebApplicationFactory. Specifically: the 409 conflict scenario — two concurrent updates, verify the second returns 409. The filter combinations on GetAll — from/to/status. The AddAttendee duplicate check via HTTP. And a test that Cancel returns 200 regardless of RowVersion.

Is this CQRS?

Logically yes — GET endpoints only read, POST/PUT/DELETE only write, they share nothing. Physically no — there's one database and no separate read model. To make it physical CQRS you'd introduce a read-optimised model, a separate database or projection, and MediatR to route commands and queries to separate handlers. At this scale that's over-engineering.

SOLID — give me a concrete example of each

Single Responsibility — each controller handles exactly one operation.

CancelEventController has one reason to change: the cancel behaviour.

Open/Closed — INotificationService. A new email implementation can be added without touching any existing class. The domain and controllers are closed to modification, open to extension via the interface.

Liskov Substitution — LoggingNotificationService is substitutable for any INotificationService implementation. Swap it for EmailNotificationService and every caller behaves correctly.

Interface Segregation — ICalendarEventRepository is focused. It doesn't have thirty methods. Attendees don't have their own repository because they don't need one.

Dependency Inversion — Controllers depend on ICalendarEventRepository, not CalendarEventRepository. The domain depends on INotificationService, not LoggingNotificationService. We depend on abstractions.

Scoped vs Singleton — which did you use and why?

DbContext is Scoped — one instance per HTTP request. This is the EF Core recommendation because DbContext is not thread-safe and tracks entity state per request. The repository is also Scoped because it takes DbContext as a constructor dependency — if the repository were Singleton it would hold a reference to a Scoped DbContext that gets disposed at the end of the first request. That's the captive dependency problem. INotificationService is Scoped for the same reason.

What would break if DbContext were Singleton?

First request completes, DbContext is disposed, second request tries to use it — ObjectDisposedException. Even before that, entity tracking would bleed across requests — changes one request made would be visible to another. DbContext is explicitly designed to be short-lived.

Why ControllerBase and not Controller?

Controller inherits from ControllerBase and adds View support — Razor views, ViewBag, ViewData, redirect-to-view helpers. This is a REST API. There are no views. Using Controller would pull in MVC view infrastructure for no reason. ControllerBase gives everything an API needs: Ok(), NotFound(), StatusCode(), model binding, [ApiController] behaviour.

How would you add domain events — fire EventCancelled and let a handler send the notification?

Entities collect domain events in a list during their operation — CalendarEvent.Cancel() adds a new EventCancelledDomainEvent to a private collection. The repository's UpdateAsync reads and clears that collection after SaveChanges, then dispatches each event through MediatR's IPublisher. A separate CancelledEventNotificationHandler handles it and calls INotificationService. This decouples the notification from the domain method — the entity doesn't know or care what happens after it's cancelled.

How would you add soft delete?

Add an IsDeleted bool property to CalendarEvent with a private setter. Only Cancel() sets it. Add a global query filter in CalendarEventConfiguration: builder.HasQueryFilter(e => !e.IsDeleted). Every SELECT automatically excludes deleted records. To query deleted records explicitly, call .IgnoreQueryFilters() on that specific query.

If you had to split into microservices, where's the boundary?

The natural split is between scheduling and notifications. The calendar API owns events and attendees. A separate notification service subscribes to events published to a message bus — EventCreated, EventCancelled — and handles email, SMS, iCal. The

boundary is clean because INotificationService already abstracts it. The only change is swapping the in-process logging stub for a message bus publisher.

Clarification Questions

Why Shouldly specifically and not xUnit assertions?

Two reasons. First, the failure message. xUnit's built-in Assert.Equal gives you "Expected: Scheduled, Actual: Cancelled". Shouldly's `calendarEvent.EventStatus.ShouldBe(EventStatus.Scheduled)` gives you a full sentence with the variable name — you know exactly what failed without reading the test code. Second, readability. `Should.Throw<ArgumentException>(act).Message.ShouldContain("End time")` chains naturally and reads like English. The alternative is FluentAssertions — same idea but recently moved to a paid model for commercial use. Shouldly is MIT-licensed.

Is MediatR paid?

No — MediatR is free and MIT-licensed. The confusion comes because Jimmy Bogard also maintains AutoMapper, which recently moved to a paid model for commercial use. They are often used together so people mix them up. MediatR has no commercial licensing restrictions.

Should HasQueryFilter be on the domain entity?

No. HasQueryFilter is EF Core configuration — it belongs in `CalendarEventConfiguration` alongside column mappings, max lengths, concurrency token, and cascade deletes. It is a persistence concern, not a domain concern. The domain entity's job is to expose the `IsDeleted` property and control how it gets set through a private setter and domain method. If you swapped EF Core for Dapper tomorrow, HasQueryFilter disappears entirely. The private setter and the domain rule stay. Domain logic is infrastructure-agnostic — HasQueryFilter is not.

Do you need a new repository for every domain entity?

No — one repository per aggregate root, not per entity. Attendee has no repository because it only exists through `CalendarEvent`. EF Core persists it automatically via the cascade relationship. If you added a new concept that stands on its own — Doctor, Room — and you need to query it independently, it gets its own repository. If you added something like `EventNote` that only makes sense as part of a `CalendarEvent`, it's a child entity — no repository, just a method like `AddNote()` on `CalendarEvent`.

Can a child entity be created separately, like Attendee?

It can be — but in this solution it's a deliberate design decision to prevent it. `Attendee.Create()` is internal. Nothing outside the assembly can call it. The only way to create an Attendee is through `CalendarEvent.AddAttendee()`, which runs the duplicate

email guard and the cancelled event guard. If Attendee had a public constructor a developer could bypass both guards entirely. The internal keyword makes that impossible at compile time. In a multi-project solution with Domain as a separate class library, internal is even tighter — Attendee.Create() would only be accessible within the Domain project itself.

Is there another way to handle concurrency besides optimistic concurrency token?

Yes — pessimistic concurrency using database locks. Instead of letting both doctors load the record freely and detecting the conflict at save time, you lock the row when the first doctor reads it. The second doctor's read blocks until the first releases the lock. In EF Core this is done with a raw SQL hint like `SELECT * FROM CalendarEvents WITH (UPDLOCK)`. The lock is held until the transaction commits or rolls back.

The trade-off: optimistic concurrency assumes conflicts are rare, no lock is taken, throughput is high. Pessimistic concurrency assumes conflicts are frequent, the row is locked on read, throughput is lower and deadlock risk exists. Optimistic is almost always right for web APIs because HTTP is stateless — you cannot hold a database lock across two separate HTTP requests. By the time Doctor B submits their form, Doctor A's request is long finished. Pessimistic concurrency makes more sense in long-running batch processes or financial systems where two writes to the same record are frequent.

Can there be a generic repository instead of one interface per aggregate root?

Yes. A generic repository with an IAggregateRoot marker interface works well for shared operations:

```
public interface IAggregateRoot { }

public interface IRepository<T> where T : IAggregateRoot
{
    Task<T?> GetByIdAsync(Guid id, CancellationToken cancellationToken = default);
    Task AddAsync(T entity, CancellationToken cancellationToken = default);
    Task UpdateAsync(T entity, CancellationToken cancellationToken = default);
    Task DeleteAsync(T entity, CancellationToken cancellationToken = default);
}
```

The problem comes when aggregates have different query needs. CalendarEvent needs GetAllAsync with date and status filters and SearchAsync. Those don't generalise to every aggregate. Most mature codebases end up with both — a generic IRepository base for common operations and a specific interface that extends it for aggregate-specific queries:

```
public interface ICalendarEventRepository : IRepository<CalendarEvent>
{
    Task<IEnumerable<CalendarEvent>> GetAllAsync(...);
    Task<IEnumerable<CalendarEvent>> SearchAsync(...);
}
```

I kept it specific here because CalendarEvent has filtering and search that don't generalise. In a larger system with many aggregates I would combine both approaches.